

Module 7 — Variables, Data Types, Constants, Expressions, Operators

Variables and Memory

- A **variable** is a named location in memory
- Each variable has:
 - a **type** (what kind of data it stores)
 - a **value** (current content)
 - a **scope** (visibility and lifetime)

```
int x = 5;
```

Memory view:

Name	Address	Value
x	0x100	5

Declaration, Initialization, Assignment

```
int x;           // declaration
x = 10;          // assignment
int y = 20;      // declaration and initialization
```

PROF

Multiple (Factorized) Declarations

```
int a, b, c;
int x = 1, y = 2;
float u = 1.0, v;
char c1 = 'A', c2 = 'B';
```

- All variables share the same type

Data Types in C

```
int a = 10;
float b = 3.14;
char c = 'A';
```

Complete Integer Types in C

C provides several integer types with different sizes:

Basic Types

```
char c = 'A';
short s = 10;
int i = 100;
long l = 1000;
long long ll = 10000;
```

Typical Sizes (may depend on system)

Type	Size (bytes)	Typical Range
char	1	-128 to 127
short	2	-32,768 to 32,767
int	4	$\sim -2 \cdot 10^9$ to $2 \cdot 10^9$
long	4 or 8	platform dependent
long long	8	very large integers

PROF

Signed vs Unsigned Variants

Each integer type can be **signed** or **unsigned**:

```
unsigned int a = 10;
signed int b = -5;
unsigned char c = 255;
```

- **signed** → can represent negative values
- **unsigned** → only non-negative values, larger positive range

Example (32-bit):

- **int**: -2,147,483,648 to 2,147,483,647
- **unsigned int**: 0 to 4,294,967,295

Unsigned Integer Behavior

```
unsigned int x = 0;
x = x - 1;
printf("%u", x);
```

Output:

```
4294967295
```

- Wrap-around or overflow (modular arithmetic)

Floating Point Range and Precision

Typical (IEEE 754):

- `float` (32-bit):
 - Range: $\sim 10^{-38}$ to 10^{38}
 - Precision: ~ 6 – 7 decimal digits
- `double` (64-bit):
 - Range: $\sim 10^{-308}$ to 10^{308}
 - Precision: ~ 15 – 16 decimal digits

Example:

```
float x = 1.23456789; // rounded
```

PROF

Integer vs Floating-Point Division

```
int a = 7 / 2;      // 3
float b = 7 / 2;    // 3.0 (integer division first!)
float c = 7.0 / 2;  // 3.5
```

Key ideas:

- If both operands are integers → **integer division**
- Result is truncated

- To obtain real division → at least one operand must be float

```
float d = (float)7 / 2; // 3.5
```

Literal constants

```
10  
3.14  
'A'
```

- Fixed values

Character Constants

```
char c = 'A';  
printf("%d", 'A'); // 65
```

- Characters are stored as integers (ASCII)

Expressions

- Combination of:
 - variables
 - constants
 - operators

PROF

Example:

```
x + y * 2
```

Important Concept

- C only has **arithmetic expressions**
- Logical expressions are a special case

```
int r = (x > 0); // 0 or 1
```

Arithmetic Operators

+ - * / %

Assignment Operator

```
x = 5;
```

- = is an **operator**
- It has the **lowest precedence in C**
- Assignment is an **expression**

Operator Precedence (Simplified)

1. !
2. * / %
3. + -
4. < <= > >=
5. == !=
6. &&
7. ||
8. =

Precedence Example

```
int x = 2 + 3 * 4 > 10 && 1;
```

Steps:

1. 3 * 4 = 12
2. 2 + 12 = 14
3. 14 > 10 → 1
4. 1 && 1 → 1

Logical Operators

Precedence

- ! > && > ||

Truth Tables

AND (&&):

A	B	A && B
0	0	0
0	1	0
1	0	0
1	1	1

OR (||):

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

NOT (!):

A	!A
0	1
1	0

Type Conversion

Implicit Conversion

```
float x = 5 + 2.5; // 5 becomes 5.0
```

Explicit Conversion (Casting)

```
float x = (float)5 / 2; // 2.5
int y = (int)3.14; // 3
```

Assignment Operators

`+=   -=   *=   /=`

Compound Statements (Blocks)

```
{  
    int x = 10;  
    int y = 20;  
}
```

- A block is treated as a single statement
-

Importance of Braces

```
if (x > 0)  
    printf("Positive");  
    printf("Done");
```

Only first statement is inside `if`

Correct:

```
if (x > 0) {  
    printf("Positive");  
    printf("Done");  
}
```

PROF

Module 8 — Basic Input and Output

printf Function

```
printf("Hello\n");
```

printf with Multiple Arguments

```
int a = 5, b = 10;
printf("a=%d b=%d\n", a, b);

float x = 3.14;
printf("x=%.2f a=%d\n", x, a);
```

printf Format Specifiers

Specifier	Meaning	Example
%d	decimal integer	printf("%d", 10);
%u	unsigned integer	printf("%u", 10);
%f	floating point	printf("%f", 3.14);
%c	character	printf("%c", 'A');
%s	string	printf("%s", "Hi");
%x	hexadecimal	printf("%x", 255);
%o	octal	printf("%o", 255);

scanf Function

```
int x;
scanf("%d", &x);
```

scanf with Multiple Arguments

```
int a, b;
scanf("%d %d", &a, &b);

float x;
int y;
scanf("%f %d", &x, &y);
```

scanf Format Specifiers

Specifier	Meaning
%d	integer

Specifier	Meaning
%u	unsigned integer
%f	float
%c	character
%s	string
%x	hexadecimal
%o	octal

Example Program

```
int a, b;
printf("Enter two numbers: ");
scanf("%d %d", &a, &b);
printf("Sum = %d\n", a + b);
```

Common Beginner Mistakes

- `=` vs `==`
- forgetting `&` in `scanf`
- missing braces
- integer division

Module 9 — Control Flow

Program Flow

- Default: sequential
- Control structures modify the execution flow
- Necessary in non-trivial algorithms

Selection Statements

if Statement

```
if (x > 0) {
    printf("Positive");
}
```

if-else Statement

```
if (x > 0) {  
    printf("Positive");  
} else {  
    printf("Non-positive");  
}
```

else-if Ladder (multiple choice)

```
if (x > 0)  
    printf("Positive");  
else if (x < 0)  
    printf("Negative");  
else  
    printf("Zero");
```

switch Statement

```
switch (x) {  
    case 1: printf("One"); break;  
    case 2: printf("Two"); break;  
    default: printf("Other");  
}
```

PROF

Iteration Statements

while Loop

Semantics:

- Condition checked **before** each iteration
- Executes **0 or more times**

while Loop (Basic Example)

```
int i = 1;  
while (i <= 5) {  
    printf("%d ", i);  
}
```

```
    i++;  
}
```

while Loop (Sum Example)

```
int i = 1, sum = 0;  
while (i <= 5) {  
    sum += i;  
    i++;  
}
```

do-while Loop

Semantics:

- Condition checked **after** execution
- Executes **at least once**

do-while Loop Example

```
int i = 1;  
do {  
    printf("%d ", i);  
    i++;  
} while (i <= 5);
```

- Executes at least once

PROF

for Loop

Semantics:

- Compact form: initialization, condition, update
- Typically used for **counter-controlled loops**

Basic

```
for (int i = 0; i < 5; i++) {  
    printf("%d ", i);  
}
```

Reverse Loop

```
for (int i = 5; i > 0; i--) {  
    printf("%d ", i);  
}
```

Loop forms

Counter-Based Loop

```
for (int i = 0; i < 5; i++) {  
    printf("%d", i);  
}
```

Loop Not Driven by Counter (Sentinel Loop)

```
int x;  
scanf("%d", &x);  
while (x != 0) {  
    printf("You entered %d\n", x);  
    scanf("%d", &x);  
}
```

- Loop stops when user enters 0

Loop with Condition on Data (guess what is printed)

```
unsigned x;  
scanf("%u", &x);  
do {  
    printf("%u ", x % 2);  
    x = x / 2;  
} while (x);
```

- Loop depends on some condition on input values

Input Loop with Break

```
while (1) {  
    int x;  
    scanf("%d", &x);
```

```
    if (x == 0)
        break;
    // does something
}
```

do-while Input Loop (what is the difference?)

```
int x;
do {
    scanf("%d", &x);
    // does something
} while (x);
```

Nested Loop

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        printf("(%d,%d) ", i, j);
    }
}
```